Huellitas Shop

Cuidado y estilo para tu mascota • Manual de Ingeniería

DOCUMENTO TÉCNICO OFICIAL DE DESARROLLO DE SOFTWARE

Manual Técnico de Plataforma y Arquitectura

Guía integral sobre arquitectura desacoplada en capas, modelos de persistencia relacional, contratos RESTful, integración con pasarelas de pago y proveedores de identidad, políticas estrictas de ciberseguridad, orquestación contenerizada con Docker y despliegue continuo.

PROYECTO
Huellitas Shop E-commerce Full Stack

VERSIÓN OFICIAL
v1.0.0 • Línea Base Producción

AUTOR Y DESARROLLADOR
Rafael Orlando Dominguez Sierra

FECHA Y ZONA HORARIA
23 de Octubre de 2026 • America/Bogota

Coral #ff6b4a | Peach #ffd7b5 | Mint #18b5a4 | Dusk #2c4b66 | Ink #1d2838 | Sand #f6ecdd | Cream #fffaf3

1 Portada y Control de Versiones

El presente manual describe de manera técnica y rigurosa cómo está construido el sistema **Huellitas Shop**. Está orientado a desarrolladores, ingenieros de operaciones (DevOps), administradores de sistemas y futuros mantenedores del software.

1.1. Historial de Cambios y Aprobación

VERSIÓN	FECHA	RESUMEN DEL CAMBIO REALIZADO	RESPONSABLE
0.1.0	17/09/2026	Definición inicial de arquitectura REST, entidades JPA y prototipo funcional frontend.	Equipo Huellitas
0.5.0	29/09/2026	Integración de Spring Security 6, JWT, pasarela PayPal y reserva atómica de stock.	Equipo Huellitas
0.8.0	04/10/2026	Implementación de política de sesión única, heartbeat y timeout por inactividad.	R. O. Dominguez S.
0.9.0	18/10/2026	Módulo de auditoría inmutable (`AuditLog`), backups SQL en caliente y reportes XLSX/PDF.	R. O. Dominguez S.
1.0.0	23/10/2026	Línea base de producción, pipeline CI/CD GitHub Actions con escaneo CodeQL SAST.	R. O. Dominguez S.

> **Aprobación Técnica**
> Documento validado y aprobado por la Dirección de Arquitectura y Aseguramiento de Calidad (QA) para la liberación de la versión 1.0.0 en el ambiente productivo.

2 Descripción General del Sistema

2.1. ¿Para qué existe este sistema? (Pregunta Guía)

Huellitas Shop existe para digitalizar y optimizar de forma integral el proceso de comercialización y administración de suministros para mascotas (alimentos, juguetes, accesorios e higiene). El software unifica una experiencia de compra omnicanal rápida y responsiva para los dueños de mascotas con un centro de control operativo y financiero de alta precisión para los administradores y personal de la tienda.

2.2. Problemas que Resuelve

- **Desincronización de Inventarios:** Previene la venta de productos sin existencias mediante transacciones concurrentes gestionadas por Spring Data JPA con bloqueo a nivel de base de datos.
- **Suplantación y Concurrencia Maliciosa:** Impone una política de *sesión única activa* con latidos periódicos (`heartbeat`), invalidando accesos duplicados.
- **Ceguera Financiera y Administrativa:** Proporciona generación instantánea de balances y reportes exportables a Microsoft Excel (`.xlsx`) y Adobe PDF (`.pdf`).
- **Falta de Trazabilidad Operativa:** Almacena de forma inmutable un registro de auditoría (`logs`) ante cualquier cambio de precio, eliminación o actualización.

2.3. Usuarios Objetivo

ROL / ACTOR	NIVEL DE ACCESO	RESPONSABILIDADES Y OPERACIONES
CUSTOMER	Cliente Externo	Exploración de catálogo, carrito reactivo, compras vía PayPal o efectivo, descarga de tickets `HS-XXXXXX`.
EMPLOYEE	Personal de Tienda	Gestión y despacho de órdenes, confirmación de pagos en efectivo/transferencia, consulta de reportes y catálogo total.
ADMIN	Administrador General	Control total: CRUD de productos y usuarios, generación y restauración de backups SQL, auditoría de eventos y analítica.

2.4. Restricciones Técnicas

- **Zona Horaria Obligatoria:** `America/Bogota` configurada en JVM, Hibernate JDBC, Jackson JSON y visualización frontend.
- **Moneda Oficial:** Pesos Colombianos (`COP`) con formateo estandarizado de moneda sin decimales ambiguos.
- **Política de Sesión:** Máximo una sola sesión abierta por usuario. Si se inicia sesión en otro dispositivo o ventana, la previa se revoca (código `SESSION_REPLACED`).
- **Inactividad:** Expiración automática tras 2 minutos sin pulsaciones de actividad o heartbeat.

3 Requisitos Técnicos

3.1. Requisitos de Hardware

RECURSO	ENTORNO DE DESARROLLO (MÍNIMO)	SERVIDOR DE PRODUCCIÓN (RECOMENDADO)
Procesador (CPU)	4 núcleos x86_64 a 2.0 GHz (Intel Core i5 / AMD Ryzen 5)	4 a 8 núcleos virtuales (vCPU) a 2.5 GHz o superior
Memoria RAM	8 GB DDR4	16 GB DDR4/DDR5 con soporte ECC
Almacenamiento	20 GB SSD libres	80 GB NVMe SSD con snapshots de almacenamiento
Red	Acceso a Internet para descarga de librerías y SDKs	Enlace simétrico dedicado de 100 Mbps o superior

3.2. Requisitos de Software y Versiones

COMPONENTE	TECNOLOGÍA / MOTOR	VERSIÓN EXACTA	FUNCIÓN EN EL SISTEMA
Motor de Base de Datos	PostgreSQL	16-Alpine	Base de datos relacional transaccional (ACID).
Servidor Web & Proxy	Nginx	1.27-Alpine	Servidor de estáticos y Reverse Proxy hacia la API REST.
Lenguaje & Runtime Backend	OpenJDK Java	Java 25 (LTS)	Plataforma de ejecución del servicio backend.
Framework Backend	Spring Boot	3.5.16	Framework MVC REST, Spring Security 6 y Spring Data JPA.
Librería JWT	JJWT (io.jsonwebtoken)	0.12.6	Firma, verificación y parsing de tokens HMAC-SHA.
Framework Frontend	React + Vite	React 18.3 / Vite 7.3	Interfaz SPA modular de alto rendimiento.
Sistema de Estilos	Tailwind CSS	4.3.3	Framework de utilidades con paleta corporativa y modo oscuro.
Orquestación	Docker & Docker Compose	Compose v2.24+	Aislamiento y despliegue unificado de contenedores.

4 Arquitectura del Sistema

El sistema implementa una arquitectura **Cliente-Servidor Desacoplada** basada en microcontenedores coordinados por Docker Compose. El frontend es una Single Page Application (SPA) que interactúa de manera 100% asíncrona mediante peticiones HTTP/JSON contra una API RESTful Stateless protegida por tokens Bearer JWT.

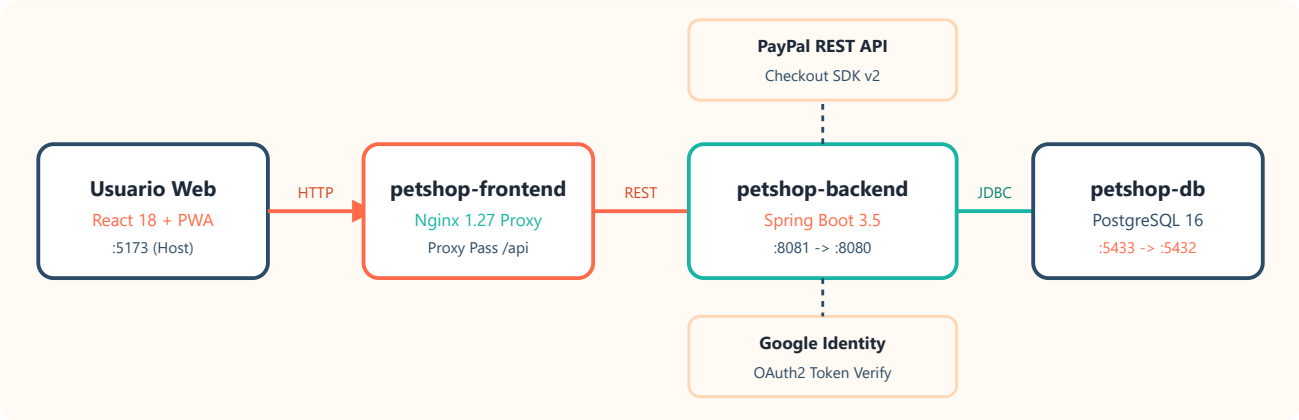


Figura 4.1: Diagrama General de Componentes e Infraestructura de Huellitas Shop

4.1. Arquitectura por Capas en el Backend

- **Controller (Controladores REST):** Expone los endpoints, maneja el protocolo HTTP y serializa/deserializa los DTOs.
- **Service (Capa de Lógica de Negocio):** Aplica reglas de stock, políticas de sesión, validación de estados y transaccionalidad (`@Transactional`).
- **Repository (Persistencia):** Interfaces Spring Data JPA con métodos derivados y optimizados.
- **Entity (Modelo de Dominio):** Clases anotadas con Jakarta Persistence (JPA) que mapean a las tablas de PostgreSQL.
- **Audit & Security:** Interceptores transversales que validan la sesión única y registran cada mutación en la base de datos de manera inmutable.

5 Módulos del Sistema

Cada módulo del sistema cuenta con una función específica, contratos de entrada y salida claros, y dependencias bien acotadas:

Módulo 1: Autenticación y Control de Sesión

CORE

Función: Registra clientes, autentica usuarios locales y con Google OAuth2, genera tokens JWT con claim de sesión única, y gestiona el cierre formal y pings de inactividad (`heartbeat`).

Entradas: `RegisterRequest`, `AuthRequest`, `GoogleAuthRequest`.

Salidas: `AuthResponse` (token JWT firmado, id, nombre, email, rol) o código 204 en logout/heartbeat.

Dependencias: `UserRepository`, `UserSessionRepository`, `ActiveSessionService`, `JwtService`, `BCryptPasswordEncoder`.

Módulo 2: Catálogo de Productos e Inventario

CATÁLOGO

Función: Permite consultar el catálogo público con filtrado y búsqueda por texto, así como el mantenimiento administrativo completo (creación, edición de precios/stock y borrado lógico/físico).

Entradas: Parámetros de consulta (`search`), `ProductRequest` con validación de precios positivos y stock.

Salidas: Lista de `ProductResponse`, objeto individual o respuesta de confirmación.

Dependencias: `ProductRepository`, `AuditLogRecorder`.

Módulo 3: Gestión de Pedidos y Tickets

TRANSACCIONAL

Función: Procesa el checkout de compras, descuenta inventario, calcula montos totales, genera comprobantes con identificador único `HS-XXXXXX` y gestiona el ciclo de vida del pedido (`PENDING`, `PAID`, `SHIPPED`, `CANCELLED`).

Entradas: `OrderRequest` (userId, lista de items con cantidad), `OrderStatusUpdateRequest`.

Salidas: `OrderResponse` con detalle de subtotal por ítem, `TicketResponse`.

Dependencias: `PetOrderRepository`, `OrderItemRepository`, `ProductRepository`, `OrderSecurity`.

Módulo 4: Pasarela de Pagos (PayPal & Efectivo)

PAGOS

Función: Orquesta la creación de órdenes en los servidores de PayPal, redirección segura a PayPal Checkout, captura de fondos autorizados y confirmación manual de pagos presenciales en efectivo.

Entradas: `orderId`, `PaymentCaptureRequest` (con `paypalOrderId`), `PaymentUpdateRequest`.

Salidas: `PaymentOrderResponse` (approveUrl), `PaymentCaptureResponse`.

Dependencias: `PayPalService`, `PayPalCheckoutSDK`, `PetOrderRepository`.

Módulo 5: Analítica y Reportes Ejecutivos

REPORTES

Función: Agrega métricas financieras (total facturado, ticket promedio, ventas por categoría y productos más vendidos) y exporta libros tabulares en formato Microsoft Excel y documentos PDF con membrete.

Entradas: `ReportFilter` (rango de fechas, categoría, estado), `ReportFormat` (`XLSX`, `PDF`).

Salidas: `ReportSummary` en JSON, o archivo binario en streaming con cabecera `Content-Disposition`.

Dependencias: `ReportService`, `Apache POI` (poi-ooxml), `LibrePDF / OpenPDF`.

Módulo 6: Auditoría, Trazabilidad y Backups

SEGURIDAD & TI

Función: Registra de manera automática e inalterable cada acción administrativa en la base de datos (`AuditLog`), proporciona consulta paginada de auditoría y permite realizar volcados y restauraciones completas en SQL.

Entradas: Eventos de Hibernate (`post-insert`, `post-update`, `post-delete`), archivo `.sql` multipart para restauración.

Salidas: Registros paginados de auditoría, archivo SQL descargable o estado de restauración.

Dependencias: `AuditLogRepository`, `HibernateAuditListener`, `BackupService`, `JdbcTemplate`.

6 Base de Datos

6.1. Modelo Entidad-Relación (MER)

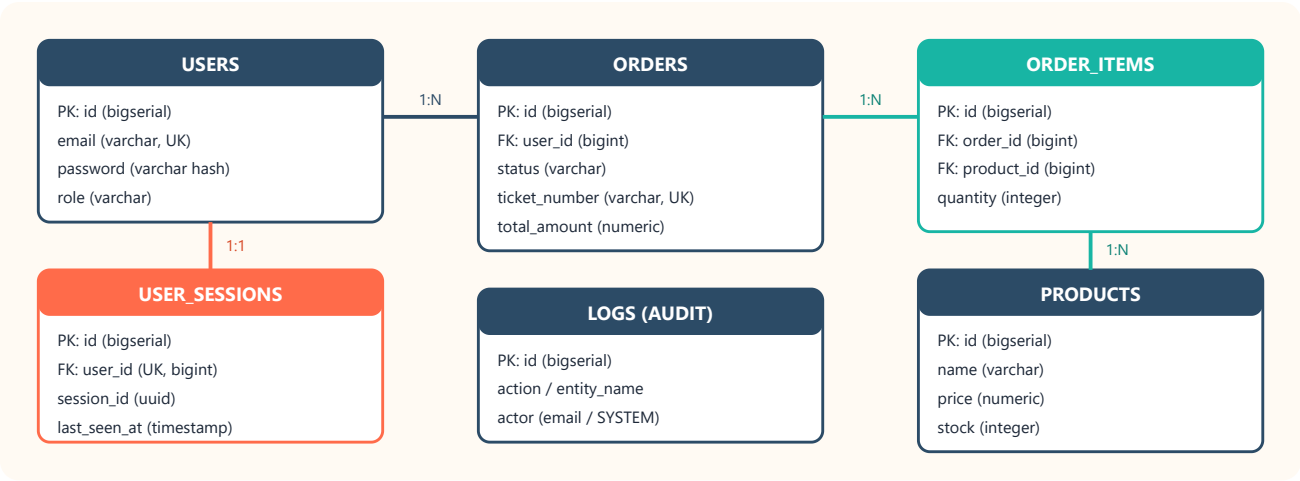


Figura 6.1: Diagrama Entidad-Relación de Huellitas Shop en PostgreSQL

6.2. Diccionario de Datos Completo

Tabla: `users`

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
`id`	BIGSERIAL	PK, NOT NULL	Identificador único y secuencial del usuario.
`full_name`	VARCHAR(255)	NOT NULL	Nombre y apellidos del usuario.
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Correo electrónico institucional o personal para autenticación.
`password`	VARCHAR(255)	NOT NULL	Hash criptográfico de la contraseña protegido con BCrypt.
`role`	VARCHAR(20)	NOT NULL	Rol de seguridad: `ADMIN`, `EMPLOYEE`, `CUSTOMER`.
`created_at`	TIMESTAMP	NOT NULL	Fecha y hora de registro oficial en zona `America/Bogota`.
`updated_at`	TIMESTAMP	NULLABLE	Fecha y hora de la última modificación del perfil.

Tabla: `products`

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
`id`	BIGSERIAL	PK, NOT NULL	Identificador único del producto en inventario.
`name`	VARCHAR(255)	NOT NULL	Nombre comercial del producto.
`description`	VARCHAR(2000)	NULLABLE	Descripción detallada, beneficios y composición.
`price`	NUMERIC(10,2)	NOT NULL	Precio unitario de venta en Pesos Colombianos (COP).
`stock`	INTEGER	NOT NULL	Cantidad de unidades físicas disponibles en bodega.
`category`	VARCHAR(100)	NULLABLE	Clasificación del producto (Perros, Gatos, Higiene, Accesorios).
`image_url`	VARCHAR(255)	NULLABLE	URL con la imagen ilustrativa del producto.
`active`	BOOLEAN	NOT NULL (def: true)	Indicador de visibilidad en el catálogo público de la tienda.

Tabla: `orders`

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
`id`	BIGSERIAL	PK, NOT NULL	Identificador numérico correlativo de la orden.
`user_id`	BIGINT	FK -> users(id), NOT NULL	Cliente que realizó la compra.
`status`	VARCHAR(30)	NOT NULL	Estado actual: `PENDING`, `PAID`, `SHIPPED`, `CANCELLED`.
`payment_method`	VARCHAR(30)	NULLABLE	Método de liquidación: `PAYPAL` o `CASH`.
`ticket_number`	VARCHAR(20)	UNIQUE, NULLABLE	Código de ticket oficial impreso (ejemplo: `HS-000105`).
`total_amount`	NUMERIC(10,2)	NOT NULL (def: 0)	Monto consolidado total de la orden en COP.
`paypal_order_id`	VARCHAR(255)	UNIQUE, NULLABLE	Identificador de transacción registrado en el API de PayPal.
`created_at`	TIMESTAMP	NOT NULL	Estampa de tiempo del momento exacto de la orden.

Tabla: `order_items`

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
`id`	BIGSERIAL	PK, NOT NULL	Identificador único del ítem del pedido.
`order_id`	BIGINT	FK -> orders(id), NOT NULL	Pedido al que pertenece esta línea.
`product_id`	BIGINT	FK -> products(id), NOT NULL	Producto físico comprado.
`quantity`	INTEGER	NOT NULL	Número de unidades adquiridas.
`unit_price`	NUMERIC(10,2)	NOT NULL	Precio unitario congelado al instante de la venta.

Tabla: `user_sessions`

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
`id`	BIGSERIAL	PK, NOT NULL	Identificador del registro de sesión.
`user_id`	BIGINT	FK -> users(id), UNIQUE	Usuario dueño de la sesión activa simultánea.
`session_id`	VARCHAR(36)	NOT NULL	UUID único de sesión inyectado en el claim del JWT.
`last_seen_at`	TIMESTAMP	NOT NULL	Marca de tiempo de la última petición o latido de heartbeat.

Tabla: `logs` (Auditoría Inmutable)

CAMPO	TIPO DE DATO	RESTRICCIONES	DESCRIPCIÓN
`id`	BIGSERIAL	PK, NOT NULL	Identificador secuencial del evento.
`action`	VARCHAR(30)	NOT NULL	Acción ejecutada: `INSERT`, `UPDATE`, `DELETE`, `LOGIN`.
`entity_name`	VARCHAR(100)	NOT NULL	Nombre de la clase o entidad afectada (`Product`, `User`, etc.).
`entity_id`	VARCHAR(100)	NULLABLE	Identificador de la fila modificada.
`actor`	VARCHAR(255)	NOT NULL	Email del usuario autenticado que ejecutó la acción o `SYSTEM`.
`details`	TEXT	NULLABLE	Detalle textual con los atributos anteriores y nuevos.
`created_at`	TIMESTAMP	NOT NULL	Fecha y hora del suceso auditado.

7 Estructura del Código

7.1. Distribución del Backend (`/backend/src/main/java/com/petshop`)

```
backend/src/main/java/com/petshop/
├── audit/                  # Listeners de Hibernate y registradores de eventos inmutables
├── config/                 # SecurityConfig, PayPalConfig, DataSeeder y SessionPolicy
├── dto/                    # Contratos de transferencia (Requests, Responses y PageResponse)
│   ├── auth/ • backup/ • log/ • order/ • payment/ • product/ • report/ • user/
│   └── controller/         # REST Controllers expuestos por Spring MVC
├── entity/                 # Entidades JPA (User, Product, PetOrder, OrderItem, UserSession, AuditLog)
├── event/                  # Eventos y escuchadores asíncronos internos (OrderPaidEvent)
├── exception/              # GlobalExceptionHandler y excepciones tipadas (BadRequest, Forbidden)
├── repository/             # Interfaces Spring Data JPA para PostgreSQL
├── security/               # Filtro JwtAuthenticationFilter, JwtService y reglas OrderSecurity
├── service/                # Servicios de negocio transaccionales (@Transactional)
│   └── report/             # ReportService y exportadores especializados Excel/PDF
└── validation/             # Validadores de política de contraseñas robustas (StrongPassword)
```

7.2. Distribución del Frontend (`/frontend/src`)

```
frontend/src/
├── components/             # Componentes modulares reutilizables
│   ├── layout/             # Navbar principal y Footer con branding
│   ├── InactivityGuard.jsx # Escucha de eventos de inactividad con modal de advertencia
│   ├── ProductCard.jsx     # Renderizado de tarjeta de catálogo con botón de carrito
│   ├── ProtectedRoute.jsx  # Guardia de rutas por rol (requireAdmin, requireStaff)
│   └── TicketModal.jsx     # Renderizado e impresión de comprobantes de venta
├── config/                 # Parámetros de inactividad y constantes de aplicación
├── context/                # Estados compartidos de React (Auth, Cart, Theme, Language)
├── hooks/                  # Custom Hooks (useIdleTimer, useSessionHeartbeat)
├── i18n/                   # Diccionarios de internacionalización (es.json y en.json)
├── pages/                  # Vistas principales (Home, Login, Register, Cart, Admin, Reports)
├── services/               # Clientes Axios con inyección automática de Bearer Token
└── utils/                  # Formateadores de moneda colombiana (COP) y helpers de blob
```

8 APIs e Integraciones

8.1. Matriz de Endpoints REST

MÉTODO	RUTA / ENDPOINT	PERMISOS REQUERIDOS	CÓDIGOS HTTP	DESCRIPCIÓN DEL ENDPOINT
POST	`/api/auth/register`	PÚBLICO	201, 400	Registro de cliente nuevo con validación de contraseña.
POST	`/api/auth/login`	PÚBLICO	200, 400	Autenticación y retorno de JWT con claim de sesión.
POST	`/api/auth/logout`	AUTENTICADO	204, 401	Cierre de sesión y revocación del identificador activo.
POST	`/api/auth/heartbeat`	AUTENTICADO	204, 401	Latido de actividad para evitar timeout de inactividad.
GET	`/api/products`	PÚBLICO	200	Listado de catálogo público con filtro de búsqueda.
GET	`/api/products/admin/all`	STAFF	200, 403	Listado de inventario total (activos e inactivos).
POST	`/api/products`	STAFF	201, 400, 403	Alta de nuevo producto en catálogo.
PUT	`/api/products/{id}`	STAFF	200, 400, 404	Modificación de precios, stock, categoría o datos.
DELETE	`/api/products/{id}`	ADMIN	204, 404	Eliminación permanente de producto.
GET	`/api/orders`	STAFF	200, 403	Consulta paginada de todos los pedidos radicados.
GET	`/api/orders/user/{userId}`	PROPIETARIO / STAFF	200, 403	Historial de compras del cliente actual.
GET	`/api/orders/{id}/ticket`	PROPIETARIO / STAFF	200, 404	Comprobante de venta digital con número oficial.
POST	`/api/orders`	PROPIETARIO / STAFF	201, 400	Radicación de pedido y reserva atómica de existencias.
PATCH	`/api/orders/{id}/status`	STAFF	200, 400	Cambio de estado del pedido (`PAID`, `SHIPPED`, etc.).
POST	`/api/payment/paypal/create-order`	PROPIETARIO / ADMIN	200, 400	Generación de orden en PayPal Checkout API v2.
POST	`/api/payment/paypal/capture`	PROPIETARIO / ADMIN	200, 400	Captura y liquidación de fondos autorizados por PayPal.
GET	`/api/reports/summary`	STAFF	200, 403	Resumen analítico con métricas e indicadores clave.
GET	`/api/reports/export`	STAFF	200, 403	Descarga directa de reporte en formato `.xlsx` o `.pdf`.

MÉTODO	RUTA / ENDPOINT	PERMISOS REQUERIDOS	CÓDIGOS HTTP	DESCRIPCIÓN DEL ENDPOINT
GET	`/api/logs`	STAFF	200, 403	Consulta paginada del registro inmutable de auditoría.
GET	`/api/backup/download`	AUTENTICADO	200, 401	Descarga del volcado SQL completo de la base de datos.
POST	`/api/backup/restore`	AUTENTICADO	200, 500	Restauración supervisada de la base de datos desde `.sql`.

9 Seguridad

> **Arquitectura de Seguridad en Profundidad**
> El sistema implementa mecanismos de defensa en múltiples capas, asegurando la confidencialidad, integridad y disponibilidad de la información conforme a las mejores prácticas de OWASP.

- **Autenticación y Criptografía:** Cifrado de contraseñas mediante `BCryptPasswordEncoder` con salt dinámico por usuario. Generación de tokens JWT firmados con clave simétrica secreta (HMAC-SHA256) de longitud segura.
- **Control de Acceso por Roles (RBAC):** Protección declarativa de métodos mediante `@PreAuthorize` con los roles `ROLE_ADMIN`, `ROLE_EMPLOYEE` y `ROLE_CUSTOMER`. Reglas de propiedad SpEL (`@orderSecurity.ownsOrder(#id, authentication)`) para evitar acceso horizontal no autorizado entre clientes.
- **Protección contra Inyección SQL (SQLi):** Consultas 100% parametrizadas a través de Hibernate ORM y Spring Data JPA, impidiendo la concatenación dinámica de sentencias SQL maliciosas.
- **Mitigación de Cross-Site Scripting (XSS):** El Virtual DOM de React escapa y neutraliza contextualmente todas las cadenas de texto renderizadas antes de insertarlas en el árbol DOM del navegador.
- **Mitigación de CSRF:** Arquitectura REST Stateless con cabecera obligatoria `Authorization: Bearer <token>` sin uso de cookies vulnerables a cross-origin.
- **Control de Concurrencia e Inactividad:** Servicio `ActiveSessionService` que fuerza la sesión única por cuenta de usuario y cierra la sesión tras 2 minutos de inactividad asistido por un `InactivityGuard` en el frontend.
- **Auditoría Inmutable:** La entidad `AuditLog` no cuenta con métodos setters ni endpoints de modificación/borrado, garantizando un registro fidedigno de auditoría.

10 Instalación y Configuración

10.1. Despliegue con Docker Compose (Recomendado)

Paso 1: Clonar el proyecto

```
git clone https://github.com/usuario/api2.git
cd api2
```

Paso 2: Configurar las Variables de Entorno (`.env`)

```
# Base de Datos PostgreSQL
POSTGRES_DB=petshop_db
POSTGRES_USER=postgres
POSTGRES_PASSWORD=TuClaveSegura123!
POSTGRES_PORT=5433

# Puertos y Red
BACKEND_PORT=8081
FRONTEND_PORT=5173

# Seguridad JWT y Admin
JWT_SECRET=dGVzdF9zZWNyZXRfa2V5X3Rlc3Rfc2VjcmV0X2tleV90ZXN0X3NlY3JldF9rZXk=
JWT_EXPIRATION_MS=86400000
ADMIN_INITIAL_PASSWORD=Admin12345!

# Pasarela PayPal y Google OAuth
PAYPAL_CLIENT_ID=sb-dummy-id
PAYPAL_CLIENT_SECRET=sb-dummy-secret
PAYPAL_MODE=sandbox
GOOGLE_CLIENT_ID=dummy-client-id.apps.googleusercontent.com
```

Paso 3: Construir y Levantar los Contenedores

```
docker compose up --build -d
```

Paso 4: Validar Estado de Salud y Puertos

```
docker compose ps
```

SERVICIO	CONTENEDOR	PUERTO HOST	PUERTO INTERNO	ESTADO
Frontend + Proxy	petshop-frontend	5173	80	Up
Backend API	petshop-backend	8081	8080	Up
PostgreSQL	petshop-db	5433	5432	Up (healthy)
Adminer	petshop-adminer	8082	8080	Up

11 Despliegue y CI/CD

11.1. Ambientes de la Solución

- **Desarrollo (Local):** Ejecución con live-reload en Vite (`npm run dev`) y Spring Boot Devtools con base de datos en Docker.
- **Pruebas e Integración Continua (CI):** Pipeline automatizado en GitHub Actions con servicios efímeros de PostgreSQL, validando la compilación, pruebas y análisis estático.
- **Producción:** Despliegue en contenedores optimizados con Nginx como proxy inverso, compresión gzip y volúmenes de datos persistentes.

11.2. Pipeline Automatizado de Integración Continua (`.github/workflows/ci.yml`)

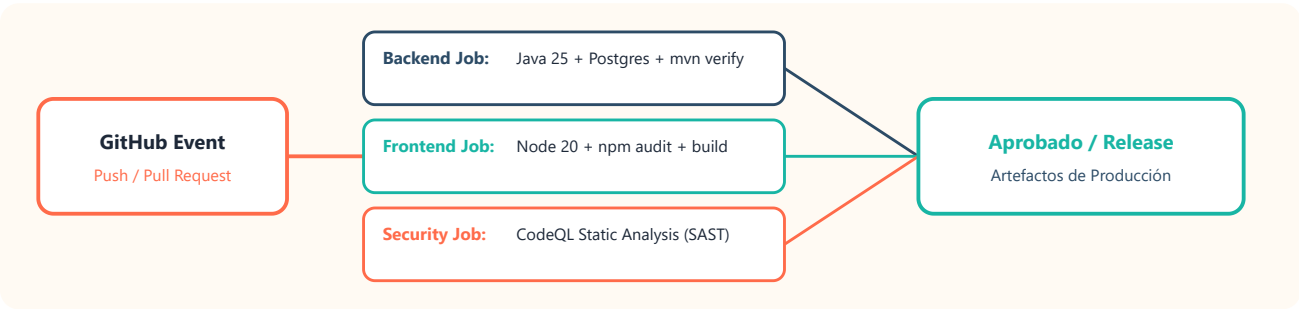


Figura 11.1: Flujo de Integración Continua en GitHub Actions

12 Mantenimiento del Sistema

12.1. Gestión y Monitoreo de Logs

Monitoreo de eventos en tiempo real mediante comandos nativos de Docker:

```
docker compose logs -f --tail=100 backend
docker compose logs -f db
```

12.2. Política de Respaldo y Recuperación (Disaster Recovery)

- **Frecuencia:** Respaldos diarios en horario de bajo tráfico (02:00 AM America/Bogota).
- **Comando de Volcado Manual en Caliente:**

```
docker exec -t petshop-db pg_dump -U postgres -d petshop_db -F c -b -v -f /var/lib/postgresql/data/backup_$(date +%Y%
```

- **Procedimiento de Restauración:** Cargar el volcado a través del endpoint administrativo `POST /api/backup/restore` o mediante el cliente oficial `pg_restore`.

12.3. Escalabilidad Horizontal y Vertical

Gracias al diseño **Stateless** de la API (la autenticación se valida en cada petición mediante el token JWT y el registro de sesión en base de datos), el servicio backend puede escalar horizontalmente en múltiples instancias detrás de un balanceador de carga Nginx o Cloud Load Balancer sin pérdida de estado.

13 Recomendaciones y Conclusiones

Buenas Prácticas para el Equipo de Ingeniería

1. **Documentación Continua:** Mantener actualizados tanto este manual técnico como los esquemas Swagger OpenAPI ante cualquier modificación de la API o la base de datos.
2. **Seguridad de Credenciales:** Nunca versionar contraseñas, tokens de PayPal o secretos JWT en Git; utilizar siempre archivos `.env` ignorados o gestores de secretos.
3. **Consistencia Horaria:** Asegurar que todo nuevo desarrollo preserve la zona horaria `America/Bogota` para evitar discrepancias en el cálculo de reportes de ventas.
4. **Pruebas Automatizadas:** Ejecutar `mvn test` y `npm run build` antes de fusionar cualquier cambio a la rama `main`.

Huellitas Shop • Plataforma E-commerce Full Stack

Manual Técnico de Software • Elaborado conforme a los estándares de desarrollo de la organización.